

Thinf Cheat Sheet

Entscheidungsprobleme

Basics

- Problem P_1 Frage mit Antwort Ja/Nein
- $\text{co-}P_1$ ist die Verneinung der Frage von P_1
- *Entscheidbar*: Es gibt ein Programm, dass auf allen Inputs terminiert und korrekte Antwort liefert
- *Unentscheidbar*: Es gibt **keinen** Algorithmus, der für jede Eingabe das richtige Ergebnis liefert.
- *Semi-Entscheidbar*: Programm terminiert auf positiven Instanzen und liefert korrektes Ergebnis, auf negativen Instanzen darf es auch nicht terminieren (falls es terminiert, muss jedoch das korrekte Ergebnis geliefert werden)
- *Nicht-Semi-Entscheidbar*: Es gibt **keinen** Algorithmus, der auf positiven Instanzen "Ja" liefert **und** auf negativen Instanzen beliebig reagieren (halt/no halt) darf.
- Entscheidungsverfahren kriegen immer nur die Instanz eines Problems, nicht mehr, nicht weniger

Reduktionen

- Reduktion $A \leq B$ bedeutet A wird auf B reduziert
- $R(x)$ transformiert Instanz x von A auf Instanz $R(x)$ von B
- Beweis: x positive Instanz von A $\iff R(x)$ positive Instanz von B#
- Umgangssprachlich $A \leq B$: B ist mindestens so schwer wie A, A ist mindestens so leicht wie B

Entscheidbarkeitsregeln:

Zwei Probleme P_1, P_2 und gültige Reduktion $P_1 \leq P_2$:

- Es gibt eine Reduktion $\text{co-}P_1 \leq \text{co-}P_2$
- Unentscheidbarkeit links $\rightarrow$ rechts: P_1 unentscheidbar $\implies P_2$ unentscheidbar
- Nicht-Semi-Entscheidbarkeit links $\rightarrow$ rechts
- (Semi-)Entscheidbarkeit rechts $\rightarrow$ links: P_2 (semi-)entscheidbar $\implies P_1$ (semi-)entscheidbar
- $P_1, \text{co-}P_1$ semi-entscheidbar $\implies P_1, \text{co-}P_1$ entscheidbar
- P_1 unentscheidbar und semi-entscheidbar $\implies \text{co-}P_1$ nicht-semi-entscheidbar
- $P_1 \leq P_2$ und $P_2 \leq P_3 \implies P_1 \leq P_3$ (Transitivität)

Abzählbarkeit

- abzählbar $\iff$ endlich oder bijektiv zu $\mathbb{N}$
- überabzählbar $\iff$ nicht abzählbar
- Überabzählbar:
 - $\mathbb{R}, \mathbb{R}^n, [0, 1]$
 - Potenzmengen: $\mathcal{P}(\mathbb{N}), \mathcal{P}(\mathbb{R})$
 - Funktionen: $\mathbb{R}^{\mathbb{N}}, \{0, 1\}^{\mathbb{N}}$, Menge aller Funktionen: $f : \mathbb{N} \rightarrow \mathbb{N}$
 - Menge aller Sprachen über Σ
 - Menge aller nicht entscheidbaren Sprachen
- Abzählbar:
 - $\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{N}^k, \mathbb{N} \times \mathbb{N}$
 - Endliche Mengen
 - Σ^* , alle regulären, kontextfreien, rekursiven Sprachen
 - Menge aller Programme, TMs, ...

Berechenbarkeit

- Es gibt überabzählbar viele Funktionen vom Typ $f : \mathbb{N} \rightarrow \{0, 1\}$

Church-Turing-These

Ein "Programm" das etwas berechnet, kann immer auch durch TM, λ -Term, RM, partiell rekursive Funktion dargestellt werden.

Turingmaschine

- Operiert auf Band, das die Eingabe und sonst Blanksymbole enthält
- Initial ist Lese/Schreibkopf auf dem ersten Symbol der Eingabe
- TM M besteht aus:
 - Q : endliche Menge von Zuständen
 - Σ : endliche Menge der Eingabesymbole
 - Γ : endliche Menge der Bandsymbole, $\Sigma \subset \Gamma$
 - Übergangsfunktion $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$
 - Startzustand $q_0 \in Q$
 - Blank Symbol: $B \in \Gamma - \Sigma$
 - Endzustände $F \subseteq Q$
- Übergang $(q, X; p, Y, D) \in \delta$
 - M liest im Zustand q auf dem Band Symbol X
 - wechselt abhängig davon in den Zustand p
 - überschreibt X durch Y
 - bewegt den Lesekopf durch D beschrieben:
 - L links
 - R rechts

- S gar nicht (stay)
- *Deterministisch* Pro (q, X) existiert nur ein Übergang $(q, X; p, Y, D) \in \delta$
- M terminiert im Zustand q wenn $q \in F$
- Konfiguration: $X_1 X_2 \dots X_{i-1} q X_i X_{i+1} \dots X_n$ bedeutet:
 - TM ist im Zustand q , Lese/Schreib-Kopf steht über X_i
- Die akzeptierte Sprache $L(M)$ von einer TM M ist rekursiv aufzählbar

Variante aus Teil 3

- Arbeitsband und Eingabeband
- $Z_0 \in \Gamma$ linkes Begrenzungssymbol auf Arbeitsband
- Z_1, Z_2 Begrenzungssymbole des Eingabewortes
- Übergang $(q, a, X; p, Y, D_E, D_A)$
 - M liest im Zustand q auf dem Eingabeband Symbol a und auf dem Arbeitsband Symbol X
 - wechselt abhängig davon in den Zustand p
 - überschreibt auf dem Arbeitsband X durch Y
 - bewegt auf dem Eingabeband den Lesekopf durch D_E
 - bewegt auf dem Arbeitsband den Lesekopf durch D_A

Registermaschinen

- m -Registermaschine besteht aus 3 Komponenten $R = (S, O, T)$
 - $S = \mathbb{N}^m$ Speicherzustände (Inhalt der m Register)
 - O Speicherbefehle
 - T Testbefehle
- Ein m -RM-Programm besteht aus Start $a \in \mathbb{N}$ und endliche Menge Anweisung:
 - Funktionsanweisung (r, f, p) : r : do f then goto p
 - Testanweisung (r, t, p, q) : r : if t then goto p else goto q
 - r ist **Kenmmarke**, p, q sind **Sprungmarken** ($\in \mathbb{N}$), Sprungmarken die keine Kennmarken sind, heißen **Endmarken**
- Befehle für ein m -RM-Programm:
 - $R(i)$ Inhalt des i -ten Registers
 - $A_i : R(i) := R(i) + 1$
 - $S_i : R(i) = R(i) - 1$
 - $t_i : \text{Ist } R(i) = 0?$
- Berechnung einer Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ auf einer m -RM ($k \leq m$):
 - Anfangszustand: $R(i) = n_i$ für $1 \leq i \leq k$, $R(i) = 0$ sonst
 - Endzustand falls Endmarke erreicht, Ergebnis von f steht in $R(1)$
 $(f(n_1, \dots, n_k) = R(1))$

- Rechnungsnotation $p : (n_1, \dots, n_m) \iff$ aktuelle Kennmarke ist p , $R(i) = n_i$ **nach** Ausführung der vorigen Befehle, vor Ausführung des current.

λ -Kalkül

- Funktionen kompakt dargestellt, zB $\lambda x. x^2 + n$ ist äquivalent zu $f(x) = x^3 + n$, wobei x eine **gebundene** Variable, n eine **freie** Variable ist (Konstante von außerhalb der Funktion)
- Auswerten durch β -Reduktion:

$$\begin{aligned} (\lambda x. y)((\lambda z. zz)(\lambda w. w)) &\rightarrow_{\beta} (\lambda x. y)((\lambda w. w)(\lambda w. w)) \\ &\rightarrow_{\beta} (\lambda x. y)(\lambda w. w) \\ &\rightarrow_{\beta} y \end{aligned}$$

- Ende wenn nach Reduktionen der Term in Normalform ist (keine Reduktionen mehr möglich)
 - Nicht jeder Term hat eine Normalform
 - Aber jede Normalform ist eindeutig (bis auf Umbenennungen)
- Boolesche Funktionen im λ -Kalkül
 - $\mathbf{T} = \lambda xy. x$
 - $\mathbf{F} = \lambda xy. y$
 - $\mathbf{if_then_else} = \lambda pxy. pxy$

Rekursive Funktionen

- Funktionen $\mathbb{N}^k \rightarrow \mathbb{N}, k \geq 0$
- Grundfunktionen
 - Konstante: $C_n^k(x_1, \dots, x_k) = n$
 - Nachfolger $S(x) = x + 1$
 - Projektion $P_i^k(x_1, \dots, x_k) = x_i \quad (1 \leq i \leq k)$
- Komposition $\circ$:
 - Linksassoziativ: $A \circ B \circ C = (A \circ B) \circ C$
 - $f(x) = g \circ h = g(h(x))$
 - bzw für Funktion g mit m Argumenten, brauchen wir $h_1 \dots h_m$:

$$f(x_1, \dots, x_k) = g \circ (h_1, \dots, h_m) = h(g_1(x_1, \dots, x_k), \dots, g_m(x_1, \dots, x_k))$$

- Rekursion mittels $\mathbf{Pr}$
 - Funktionen g mit k , h mit $k + 2$ Argumenten, dann ist $f = \mathbf{Pr}(g, h) : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ definiert:

$$\begin{array}{ll} f(\underline{0}, x_1, \dots, x_k) = g(x_1, \dots, x_k) & \text{Base Case} \\ f(m + 1, x_1, \dots, x_k) = h(m, f(m, x_1, \dots, x_k), x_1, \dots, x_k) & \text{Rekursion} \end{array}$$

- g definiert also den Base Case
- $f(m+1)$ wird definiert durch die Funktion h die zusätzlich zu den

standard Argumenten auch noch μ und das Ergebnis des vorherigen Rekursionsschritt bekommt

- Minimierungen $\mu_0, \bar{\mu}$ einer Funktion g mit $k + 1$ Argumenten:
 - $f = \mu_0 g$ dann ist $f(x_1, \dots, x_k) = \min_{y \geq 0} [g(y, x_2, \dots, x_k) = 0]$
 - $f = \bar{\mu} g$ dann ist $f(x_1, \dots, x_k, x_{k+1}) = \min_{y \geq 0} [g(y, x_2, \dots, x_{k+1}) = x_1]$
- Funktionen mit **Pr**, $\circ$ sind **primitiv-rekursive Funktionen**.
- Funktionen mit **Pr**, $\circ$, $\mu_0, \bar{\mu}$ sind **partielle rekursive Funktionen**.
- Nicht alle totalen berechenbaren Funktionen sind primitiv rekursiv.

Satz von Rice

- Betrachtet Eigenschaften von rekursiv aufzählbaren Sprachen (also Sprache von TM)
- "Jede nicht triviale Eigenschaft einer rekursiv aufzählbaren Sprache ist unentscheidbar"
- Eine Eigenschaft ist trivial falls:
 - Alle rekursiv aufzählbaren Sprachen diese Eigenschaft besitzen ODER
 - keine rekursiv aufzählbare Sprache diese Eigenschaft besitzt
- Bezieht sich **ausschließlich** auf **extensionale** Eigenschaften
 - extensional: das "Dargestellte" (Sprachen, Probleme)
 - intensional: die "Darstellungsmittel" (TMs, PR, RM)
 - Z.b. "Berechnet TM Funktion f für Inputs 0...1000 korrekt" ist **intensional**

Formale Sprachen

Eine formale Sprache heißt:

- rekursiv aufzählbar* wenn Sie von einer Turingmaschine akzeptiert wird
- kontextsensitiv* wenn Sie von einem LBA akzeptiert wird
- kontextfrei* wenn sie von einem Kellerautomaten akzeptiert wird (Stack)
- regulär* wenn Sie von einem endlichen Automaten akzeptiert wird

Typ	Name	Grammatik	Produktionen	Akzeptor	Entscheidbarke
3	Regulär	Reguläre Grammatik	$A \rightarrow Ba \mid a \mid \epsilon$ $A \rightarrow aB \mid a \mid \epsilon$	DFA / NFA / Regex	Mitgliedschaft ✓ Leerheit ✓ Äquivalenz ✓
2	Kontextfrei	Kontextfreie Grammatik	links genau eine Variable	Keller	Mitgliedschaft ✓ Leerheit ✓ Äquivalenz ✗
1	Kontextsensitiv	Kontextsens. Grammatik	$\alpha A \beta \rightarrow \alpha \gamma \beta$	LBA	Mitgliedschaft ✓ Leerheit ?

Typ	Name	Grammatik	Produktionen	Akzeptor	Entscheidbarke
0	Rekursiv aufzählbar	Unbeschränkte Grammatik	mindestens ein Terminal	TM	Mitgliedschaft semi-✓ Leerheit ✗
—	Rekursiv	—		entscheidende TM	Mitgliedschaft ✓ Komplement ✓

- Falls für eine Grammatik $|\alpha| \leq |\beta|$ gilt, dann ist diese monoton zB k.s. Grammatik.
- Monoton und kontextsensitiv dürfen $S \rightarrow \epsilon$ falls S nie rechts vorkommt
- Alle Sprachfamilien $\mathcal{L}_i$ ($i \in 0..3$) sind gegenüber Vereinigung, Konkat, Stern abgeschlossen.
- Sprachen $\mathcal{L}_i, i \in \{0, 2, 3\}$ sind gegenüber beliebigen Homomorphismen abgeschlossen, $\mathcal{L}_1$ nur wenn $\forall a \in \Sigma : h(a) \neq \epsilon$

$$\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_{rec} \subsetneq \mathcal{L}_0$$

Pumping Lemma

- Pumping Lemma um Sprache als regulär zu beweisen
 - Jedes Wort einer regulären Sprache kann in xyz zerlegt werden, sodass $|xy| \leq m$ und $|y| > 0$ gilt.
 - Das Pumping Lemma besagt, dass dann auch $w_i = xy^iz$ für alle $i \geq 0 \in L$ sein muss.

Grammatiken

- Grammatik $G = (N, T, P, S)$
 - N endliche Menge von Variablen (Nichtterminal)
 - T endliche Menge von Terminalsymbolen
 - P Produktionen
 - $S \in N$ Startsymbol
- Grammatiken erzeugen Sprachen $L(G)$
- G_1 und G_2 sind äquivalent falls $L(G_1) = L(G_2)$

Komplexitätstheorie

Reduktionen

- Wie für Entscheidbarkeit
- Zwei Probleme P_1, P_2 mit $P_1 \leq P_2$
 - $P_2 \in NP \implies P_1 \in NP$
 - $P_2 \in P \implies P_1 \in P$
 - P_1 NP-schwer $\implies P_2$ NP-schwer

- $P_1 \in NP \wedge P_1 \text{ NP-schwer} \implies P_1 \text{ NP-complete}$
- $P \subset NP$

NP Definition

Neben Reduktion, zwei Wege um $P \in NP$ zu zeigen

Zertifikate

- NP Membership kann über Zertifikat gezeigt werden
- Zertifikate beziehen sich auf positive Instanzen
- Zertifikat Relation R besteht aus (I, C) positiver Instanz und Zertifikat Tupel
- R muss polynomiell balanciert sein: Zertifikat muss polynomiell kleiner sein als Instanz
- R muss polynomiell entscheidbar sein: Es muss einen Algo geben der in polynomieller Zeit prüft ob I mit Zertifikat C JA gibt

Choice Programm

- SIMPLE Programm + choice Befehl
- `choice(assignment1, assignment2)` teilt Berechnung in zwei Zeige die parallel laufen

NP Vollständigkeit Spezialfälle

- Spezialfälle ist selbe Frage, aber auf eingeschränktem Input
- Sei P ein Problem und P' ein Spezialfall von P , dann:
 - $P' \text{ NP-schwer} \implies P \text{ NP-schwer}$
 - $P \in NP \implies P' \in NP$

Weitere Komplexitätsklassen

- L log space
 - Problem kann mit $c \log |I| + d$ Arbeitsspeicher gelöst werden
 - $L \subseteq P$
- PSPACE
 - Programm kann mit polynomiellen Speicherbedarf gelöst werden
 - PSPACE Probleme sind *intractable*
 - $NP \subseteq PSPACE$
 - $P \subseteq PSPACE$
- EXPTIME
 - Es gibt ein SIMPLE Programm und Laufzeit ist $O(2^{|I|^k})$
 - $PSPACE \subseteq EXPTIME$
 - $P \subsetneq EXPTIME$

Formale Korrektheit

Syntax

Grammatik die einen korrekten Programmsyntax beschreibt:

$$\begin{aligned} \text{Programm} &\rightarrow \text{"skip"} \\ &| \text{"abort"} \\ &| \text{Var " := " Term} \\ &| \text{Programm ";" Programm} \\ &| \text{"if" Term "then" Programm "else" Programm "fi"} \\ &| \text{"while" Term "do" Programm "od"} \\ \\ \text{Term} &\rightarrow \text{Var} \mid \text{Const} \mid \text{UnOp Term} \mid \text{"(" Term BinOp Term ")"} \\ \text{UnOp} &\rightarrow \text{"+"} \mid \text{"-"} \mid \text{"¬"} \\ \text{BinOp} &\rightarrow \text{"+"} \mid \text{"-"} \mid \text{"*"} \mid \dots \\ \text{Const} &\rightarrow \text{"0"} \mid \text{"1"} \mid \text{"2"} \mid \dots \end{aligned}$$

Zustände

- Konfiguration (Π, σ) wobei Π ein Programm ist und σ ein Snapshot des Speichers ist
- Übergangsregeln:
- $[\text{skip}]\sigma = \sigma$
- $[v := e]\sigma = \sigma'$ wobei

$$\sigma' = \begin{cases} [e]\sigma & \text{wenn } w = v \\ \sigma(w) & \text{wenn } w \neq v \end{cases}$$

- $[\Pi; \Omega]\sigma = [\Omega][\Pi]\sigma$

$$[\text{if } e \text{ then } \Pi \text{ else } \Omega \text{ fi}]\sigma = \begin{cases} [\Pi]\sigma & \text{wenn } [e]\sigma \neq 0 \\ [\Omega]\sigma & \text{wenn } [e]\sigma = 0 \end{cases}$$

$$[\text{while } e \text{ do } \Pi \text{ od}]\sigma = \begin{cases} [\text{while } e \text{ do } \Pi \text{ od}][\Pi]\sigma & \text{wenn } [e]\sigma \neq 0 \\ \sigma & \text{wenn } [e]\sigma = 0 \end{cases}$$

Korrektheitsausagen

- Zwei Korrektheitsbegriffe
 - partiell: Falls zulässige Eingabe und das Programm terminiert, dann ist das Ergebnis richtig
 - total: Falls die Eingabe zulässig ist, dann muss das Programm terminieren und richtiges Ergebnis liefern

Hoare-Kalkül

- $F \left[\begin{smallmatrix} v' \\ v \end{smallmatrix} \right]$ bedeutet v' ersetzt v in F

wp / wlp / sp

wp(C, Q)

- stärkste notwendige Bedingung, damit:
 - C **terminiert**
 - und Q danach gilt

wlp(C, Q)

- stärkste notwendige Bedingung, damit:
 - falls C terminiert, Q gilt
 - Terminierung wird NICHT garantiert

Unterschiede

- wp berücksichtigt Terminierung
- wlp ignoriert Terminierung
- Bei terminierenden Programmen gilt: wp = wlp

sp(P, C)

- stärkste Aussage über den Endzustand
- die nach Ausführung von C aus einem Zustand mit P gilt

Intuition

- wp: rückwärts rechnen (vom Ziel zur Bedingung)
- sp: vorwärts rechnen (vom Start zum Ergebnis)

Definitionen

Regel	wlp/wp	sp
skip	$wlp(\text{skip}, Q) = Q$	$sp(F, \text{skip}) = F$
Zuweisung: $x := e$	$wlp(v := e, Q) = Q \left[\begin{smallmatrix} e \\ v \end{smallmatrix} \right]$	$sp(F, v := e) = \exists v' (F \left[\begin{smallmatrix} v' \\ v \end{smallmatrix} \right] \wedge v = e \left[\begin{smallmatrix} v' \\ v \end{smallmatrix} \right])$
Sequenz $C_1; C_2$	$wlp(C_1; C_2, Q) = wlp(C_1, wp(C_2, Q))$	$sp(F, C_1; C_2) = sp(sp(F, C_1), C_2)$
if b then C_1 else C_2	$wlp(\text{if } b \text{ then } C_1 \text{ else } C_2, Q) = (e \implies wlp(C_1, Q)) \wedge (\neg e \implies wlp(C_2, Q))$	$sp(F, \text{if } e \text{ then } C_1 \text{ else } C_2 \text{ fi}) = sp(F \wedge e, C_1) \vee sp(F \wedge \neg e, C_2)$
abort	$wlp(\text{abort}, Q) = 1, wp(\text{abort}, Q) = 0$	$sp(F, \text{abort}) = 0$

- $\{F\} \Pi \{G\}$

- Sei $F' = wlp(\Pi, G)$ bzw. $wp(\Pi, G)$
- falls $F \implies F'$ dann ist das Programm korrekt
- Sei $G' = sp(F, \Pi)$
- falls $G' \implies G$ dann ist das Programm korrekt

Annotierungsregeln

Regel	Original	Annotiert
Zuweisung (up)	$v := e$ $\{F\}$	$\left\{ F \begin{bmatrix} e \\ v \end{bmatrix} \right\} \text{zw} \uparrow$ $v := e$ $\{F\}$
Zuweisung' (down)	$\{F\}$ $v := e$	$\{F\}$ $v := e$ $\{\exists v' (F \begin{bmatrix} v' \\ v \end{bmatrix} \wedge v = e \begin{bmatrix} v' \\ e \end{bmatrix})\} \text{zw} \downarrow$
Zuweisung'' (down) Falls v nicht frei in F und e	$\{F\}$ $v := e$	$\{F\}$ $v := e$ $\{F \wedge v = e\} \text{zw} \downarrow$
If (down)	$\{F\}$ if e then ... $\{G_1\}$ else ... $\{G_2\}$ fi	$\{F\}$ if e then $\{F \wedge e\}$ if $\downarrow$... $\{G_1\}$ else $\{F \wedge \neg e\}$ if $\downarrow$... $\{G_2\}$ fi $\{G_1 \vee G_2\}$ fi $\downarrow$
If' (up) geht auch mit $\{(e \wedge F) \vee (\neg e \wedge G)\}$ if $\uparrow$	if e then $\{F\}$... else $\{G\}$	$\{(e \implies F) \wedge (\neg e \implies G)\} \text{if} \uparrow$ if e then $\{F\}$... else $\{G\}$

Regel	Original	Annotiert
If' (up)	<pre> if e then ... else ... fi $\{G\}$ </pre>	<pre> if e then ... $\{G\}$ fi $\uparrow$ else ... $\{G\}$ fi $\uparrow$ fi $\{G\}$ </pre>
While mit Invariante	<pre> while e do ... od </pre>	<pre> $\{Inv\}$ while e do $\{Inv \wedge e\}$... $\{Inv\}$ od $\{Inv \wedge \neg e\}$ </pre>
While mit Invariante + Variante (totale Korrektheit)	<pre> while e do ... od </pre>	<pre> $\{Inv\}$ while e do $\{Inv \wedge e \wedge t = t_0\}$... $\{Inv \wedge 0 \leq t < t_0\}$ od $\{Inv \wedge \neg e\}$ </pre>

Was muss man zeigen?

- $\text{Pre} \implies$ erste Annotation
- jeweils zwei benachbarte Annotationen $A \implies B$ (**direkt** über/untereinander)
- Letzte Annotation $\implies$ Post